🤖 Complete AI Agents Cheat Sheet

From Basics to Cutting-Edge Research

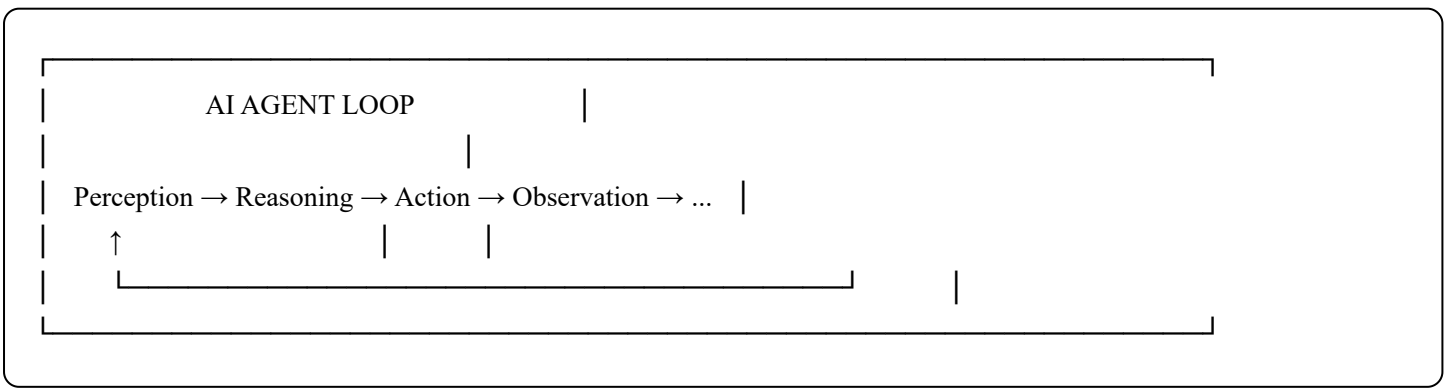
TABLE OF CONTENTS

---

1. WHAT IS AN AI AGENT?

Definition

An **AI Agent** is a system that perceives its environment, makes decisions, and takes actions to achieve specific goals — often autonomously and over multiple steps.



Agent vs. Chatbot vs. LLM

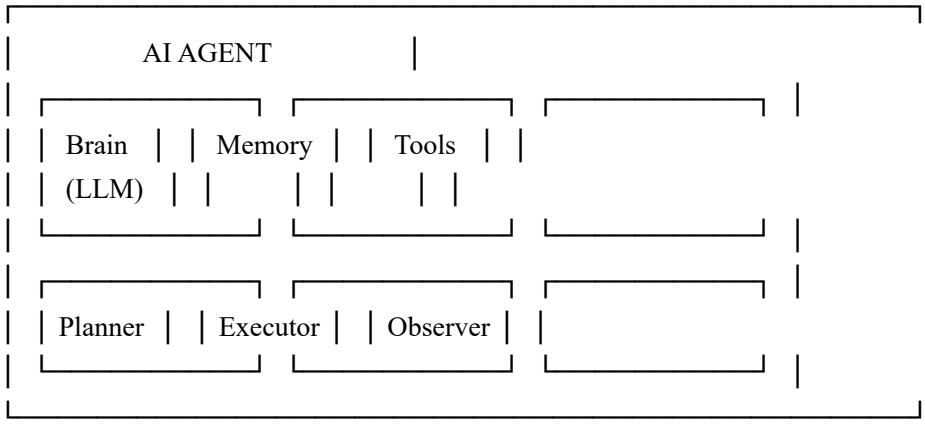
Feature	LLM	Chatbot	AI Agent
Multi-step tasks	❌	❌ (mostly)	✅
Tool/API use	❌	Limited	✅ Core feature
Memory	Context only	Session only	Short + Long term
Planning	Prompt-based	None	Explicit planning
Autonomy	None	None	High
Feedback loops	None	None	✅ Self-correcting

The Agent Paradigm Shift

Old: User → Prompt → LLM → Response
New: User → Goal → Agent → [Plan → Tools → Reflect → Iterate] → Result

---

2. CORE COMPONENTS OF AN AGENT



2.1 The Brain (LLM Core)

The reasoning engine. Popular choices:

- **GPT-4o / GPT-4-turbo** — Strong tool use, large context
- **Claude 3.5 Sonnet/Opus** — Strong instruction following, safe
- **Gemini 1.5 Pro** — 1M context, multimodal
- **Llama 3.1 405B** — Open source, self-hostable
- **Mistral Large** — Efficient, European data compliance

- **DeepSeek-V3/R1** — Strong reasoning, cost-effective

2.2 The Perception Module

What the agent can "see":

- Text (prompts, documents, chat history)
- Images / Video (multimodal agents)
- Structured data (JSON, CSV, database rows)
- Code (execution results, stack traces)
- Web pages (HTML/scraped content)
- Sensor data (IoT, robotics)

2.3 The Action Space

What the agent can "do":

- **Information**: Web search, document retrieval, database query
- **Computation**: Code execution, math, data analysis
- **Communication**: Send email, post to Slack, call APIs
- **World**: Browser control, OS interaction, robot movement
- **Memory**: Store, retrieve, update memory
- **Meta**: Spawn sub-agents, delegate tasks

---

3. TYPES OF AGENTS

3.1 By Architecture

Simple Reflex Agent

Responds to current percept only — no memory or planning.

```python
```

```
def simple_reflex_agent(percept: str, rules: dict) -> str:
    """Maps percept directly to action via condition-action rules."""
    for condition, action in rules.items():
        if condition in percept:
            return action
    return "default_action"

rules = {
    "user angry": "escalate_to_human",
    "payment failed": "retry_payment",
    "account locked": "send_reset_email"
}
```

Model-Based Reflex Agent

Maintains internal state about the world.

```
python

class ModelBasedAgent:
    def __init__(self):
        self.world_model = {}    # Internal state
        self.history = []

    def update_model(self, percept):
        """Update internal world model based on new observation."""
        self.world_model.update(percept)
        self.history.append(percept)

    def act(self, percept):
        self.update_model(percept)
        # Decision based on full internal state
        return self.decide(self.world_model)

    def decide(self, state):
        if state.get("task_complete"):
            return "finish"
        return "continue"
```

Goal-Based Agent

Has explicit goals it works to achieve.

```
python
```

```
class GoalBasedAgent:
    def __init__(self, goal: str):
        self.goal = goal
        self.current_state = {}

    def is_goal_met(self) -> bool:
        return self.evaluate_goal(self.current_state, self.goal)

    def plan_next_action(self) -> str:
        if self.is_goal_met():
            return "DONE"
        return self.search_action_space(self.current_state, self.goal)
```

Utility-Based Agent

Maximizes a utility/reward function.

python

```
class UtilityAgent:
    def evaluate_action(self, action, state) -> float:
        cost = self.compute_cost(action)
        expected_reward = self.expected_outcome_value(action, state)
        risk = self.compute_risk(action)
        return expected_reward - cost - risk * self.risk_aversion

    def best_action(self, possible_actions, state) -> str:
        return max(possible_actions,
                   key=lambda a: self.evaluate_action(a, state))
```

Learning Agent

Improves over time based on feedback.

python

```
class LearningAgent:
    def __init__(self, tools):
        self.tools = tools
        self.success_counts = {t: 0 for t in tools}
        self.total_counts = {t: 0 for t in tools}

    def select_tool(self) -> str:
        # UCB (Upper Confidence Bound) selection
        import math
        total = sum(self.total_counts.values()) + 1
        ucb_scores = {}
        for tool in self.tools:
            n = self.total_counts[tool] + 1
            mu = self.success_counts[tool] / n
            ucb_scores[tool] = mu + math.sqrt(2 * math.log(total) / n)
        return max(ucb_scores, key=ucb_scores.get)

    def update(self, tool: str, success: bool):
        self.total_counts[tool] += 1
        if success:
            self.success_counts[tool] += 1
```

3.2 By Domain

Agent Type	Description	Example Use Cases
Coding Agent	Writes, executes, debugs code	GitHub Copilot, Devin, SWE-bench
Research Agent	Searches, synthesizes, cites sources	Deep Research, Perplexity
Browser Agent	Controls web browsers	Operator, Browser-Use
Data Agent	Queries DBs, analyzes data, visualizes	Text-to-SQL, Analyst agents
Voice Agent	Speaks + listens in real-time	Customer service bots
Robotic Agent	Controls physical actuators	Boston Dynamics, RT-2
Game Agent	Plays and masters games	AlphaGo, OpenAI Five, SIMA
Science Agent	Runs experiments, proposes hypotheses	AlphaFold, FunSearch

4. AGENT REASONING & PLANNING PATTERNS

4.1 Chain-of-Thought (CoT)

Forces the model to reason step-by-step before answering.

```
python

cot_prompt = """
Problem: {problem}

Let me think step by step:
1. First, I need to understand what is being asked...
2. Then, I'll identify the relevant information...
3. Next, I'll work through the logic...
4. Finally, I'll arrive at the answer...

Answer:
"""
```

4.2 ReAct (Reason + Act)

The foundational agent pattern. Interleaves reasoning traces with tool calls.

```
Thought: I need to find the current stock price of Apple
Action: search("AAPL stock price today")
Observation: AAPL is trading at $189.30 as of 2:45 PM EST

Thought: Now I need to compare with last week's price
Action: search("AAPL stock price one week ago")
Observation: AAPL was trading at $183.20 one week ago

Thought: I can now calculate the percentage change
Action: calculate((189.30 - 183.20) / 183.20 * 100)
Observation: 3.33%

Final Answer: Apple stock has increased by 3.33% over the past week.
```

```
python
```

```
from langchain.agents import create_react_agent
from langchain import hub
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0)
tools = [search_tool, calculator_tool]
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, tools, prompt)
# The agent automatically loops: Thought → Action → Observation
```

4.3 Plan-and-Execute

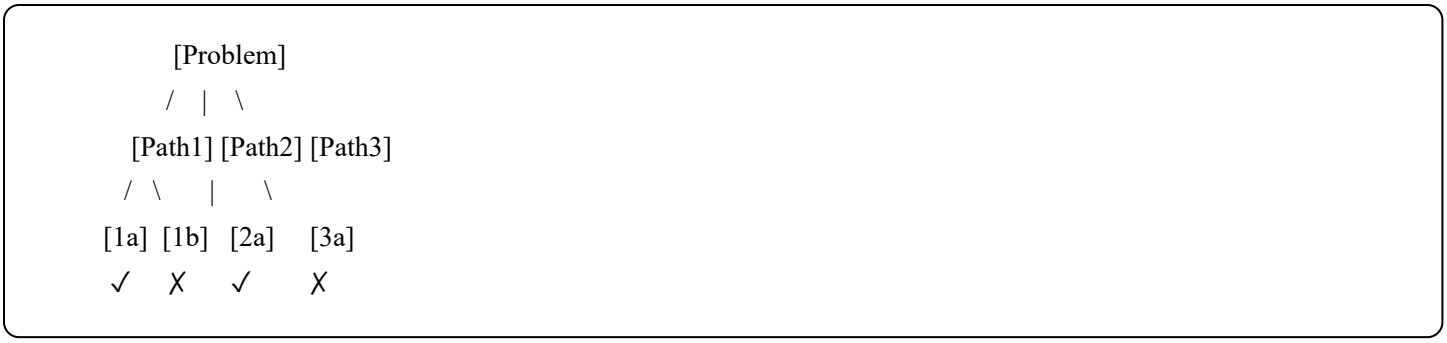
Separate planning and execution phases. Better for long-horizon tasks.

python

```
class PlanAndExecuteAgent:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools

    def run(self, goal: str):
        # Step 1: Plan
        plan = self.generate_plan(goal)
        results = []

        # Step 2: Execute each step
        for i, step in enumerate(plan):
            result = self.execute_step(step, results)
            results.append({"step": step, "result": result})

            # Step 3: Optionally replan based on results
            if self.needs_replanning(results):
                plan = self.replan(goal, results)

        return self.synthesize_results(results)

    def generate_plan(self, goal: str) -> list:
        plan_prompt = f"""
Goal: {goal}
Create a numbered step-by-step plan to achieve this goal.
Each step should be specific and actionable.
"""
        response = self.llm.invoke(plan_prompt)
        return self.parse_steps(response)

    def needs_replanning(self, results: list) -> bool:
        """Check if results suggest the plan needs updating."""
        if not results:
            return False
        last_result = results[-1]["result"]
        return "unexpected" in last_result.lower() or "error" in last_result.lower()
```

4.4 Tree of Thoughts (ToT)

Explores multiple reasoning paths simultaneously, like a tree search.



python

```
class TreeOfThoughtsAgent:
    def __init__(self, llm, branching_factor=3, depth=3):
        self.llm = llm
        self.b = branching_factor
        self.d = depth

    def generate_thoughts(self, state: str) -> list[str]:
        """Generate b possible next thoughts from current state."""
        prompt = f"""
Current state: {state}
Generate {self.b} distinct possible next steps/thoughts.
Format: numbered list
"""
        response = self.llm.invoke(prompt)
        return self.parse_thoughts(response)

    def evaluate_thought(self, thought: str) -> float:
        """Score: sure=1, likely=0.5, impossible=0."""
        prompt = f"""
Evaluate this reasoning step: {thought}
Rate as: 'sure' (1.0), 'likely' (0.5), or 'impossible' (0.0)
Return just the float score.
"""
        return float(self.llm.invoke(prompt).strip())

    def solve(self, problem: str) -> str:
        """BFS through thought tree."""
        frontier = [(problem, 0)]

        while frontier:
            state, depth = frontier.pop(0)
            if depth >= self.d:
                return state

            thoughts = self.generate_thoughts(state)
            scored = [(t, self.evaluate_thought(t)) for t in thoughts]

            # Prune low-scoring branches (beam search)
            good_thoughts = sorted(
                [(t, s) for t, s in scored if s > 0.3],
                key=lambda x: -x[1]
            )[:2]  # Keep top 2

            for thought, score in good_thoughts:
                frontier.append((state + "\n" + thought, depth + 1))
```

```
    return "No solution found"
```

4.5 Reflection & Self-Critique (Reflexion)

Agent reflects on its mistakes and improves over trials.

```python
```

```
class ReflexionAgent:
    def __init__(self, llm):
        self.llm = llm
        self.memory = []  # Stores past reflections

    def act(self, task: str) -> str:
        """Execute task with reflection history."""
        reflection_context = "\n".join(self.memory[-3:])

        prompt = f"""
Past learnings:
{reflection_context}

Task: {task}

Execute the task, applying any lessons from past attempts:
"""
        return self.llm.invoke(prompt)

    def reflect(self, task: str, action: str, outcome: str) -> str:
        """Generate reflection on what went wrong/right."""
        reflection_prompt = f"""
Task I tried: {task}
What I did: {action}
Result: {outcome}

What went wrong? What should I do differently next time?
Be specific and actionable.
"""
        reflection = self.llm.invoke(reflection_prompt)
        self.memory.append(f"Lesson: {reflection}")
        return reflection

    def run_with_reflection(self, task: str, max_trials: int = 3):
        for trial in range(max_trials):
            result = self.act(task)
            if self.is_successful(result, task):
                return result
            self.reflect(task, result, "failed")
            print(f"Trial {trial+1} failed, reflecting...")
        return result

    def is_successful(self, result: str, task: str) -> bool:
        eval_prompt = f"Task: {task}\nResult: {result}\nWas this successful? Answer yes/no."
        return "yes" in self.llm.invoke(eval_prompt).lower()
```

4.6 LATS (Language Agent Tree Search)

Combines MCTS with LLM agents — principled exploration (2023/2024).

```python
```

```
# LATS = Tree of Thoughts + Reflexion + Monte Carlo Tree Search
# UCT Score: V(s) + c * sqrt(ln(N_parent) / N(s))
# V(s) = average value of node (from reflection/evaluation)
# N(s) = visit count

import math

class LATSNode:
    def __init__(self, state: str, parent=None):
        self.state = state
        self.parent = parent
        self.children = []
        self.visits = 0
        self.value = 0.0
        self.reflections = []

    def uct_score(self, c=1.4) -> float:
        if self.visits == 0:
            return float('inf')
        parent_visits = self.parent.visits if self.parent else 1
        exploitation = self.value / self.visits
        exploration = c * math.sqrt(math.log(parent_visits) / self.visits)
        return exploitation + exploration

class LATSAgent:
    def __init__(self, llm, tools, n_simulations=10):
        self.llm = llm
        self.tools = tools
        self.n_sim = n_simulations

    def search(self, task: str) -> str:
        root = LATSNode(task)

        for _ in range(self.n_sim):
            # 1. Selection: traverse tree by UCT
            node = self.select(root)

            # 2. Expansion: generate child thoughts
            children = self.expand(node)

            # 3. Simulation: rollout to terminal state
            value = self.simulate(children[0])

            # 4. Backpropagation: update values up the tree
            self.backpropagate(children[0], value)
```

```
        # Return best path
        return self.best_path(root)

    def select(self, node: LATSNode) -> LATSNode:
        while node.children:
            node = max(node.children, key=lambda n: n.uct_score())
        return node

    def expand(self, node: LATSNode) -> list:
        thoughts = self.generate_thoughts(node.state)
        for thought in thoughts:
            child = LATSNode(node.state + "\n" + thought, parent=node)
            node.children.append(child)
        return node.children

    def simulate(self, node: LATSNode) -> float:
        """Evaluate state value."""
        eval_prompt = f"Evaluate progress toward goal:\n{node.state}\nScore 0-1:"
        score = float(self.llm.invoke(eval_prompt).strip())
        return score

    def backpropagate(self, node: LATSNode, value: float):
        while node:
            node.visits += 1
            node.value += value
            node = node.parent
```

4.7 Self-Ask with Search

```
Question: What is the birthplace of the director of Oppenheimer?

Are follow-up questions needed? Yes
Follow-up: Who directed Oppenheimer?
Intermediate answer: Christopher Nolan

Follow-up: Where was Christopher Nolan born?
Intermediate answer: Westminster, London, England

Final answer: Westminster, London, England
```

4.8 Scratchpad Pattern

Internal working memory for multi-step reasoning.

```
python
```

```
SCRATCHPAD_PROMPT = """
You have a scratchpad to work through problems.

<scratchpad>
[Use this space to: brainstorm, draft, calculate, plan]
</scratchpad>

<answer>
[Put only the final answer here]
</answer>

Task: {task}
"""
```

Pattern Comparison

Pattern	Pros	Cons	Best For
ReAct	Simple, interpretable	No backtracking	Most tasks
Plan-Execute	Structured, predictable	Rigid, hard to adapt	Long-horizon tasks
ToT	Explores many paths	Expensive (many LLM calls)	Hard reasoning
Reflexion	Learns from mistakes	Slow, multi-trial needed	Error-prone tasks
LATS	Principled exploration	Very expensive	Optimization problems
Self-Ask	Good for multi-hop	Limited action space	Research QA

---

5. MEMORY SYSTEMS

```
┌──────────────────────────────────────────────────┐
│              AGENT MEMORY TYPES                  │
├──────────────┬───────────────────────────────────┤
│ Type         │ Description                       │
├──────────────┼───────────────────────────────────┤
│ Sensory      │ Raw input buffer, immediate context │
│ Short-Term   │ Current session working memory    │
│ Long-Term    │ Persistent across sessions        │
│ Episodic     │ Specific past events              │
│ Semantic     │ General world facts               │
```

Procedural	How-to knowledge, skills

5.1 In-Context Memory (Working Memory)

The model's context window — fast but ephemeral.

```python
class ConversationMemory:
    def __init__(self, max_tokens=8000):
        self.messages = []
        self.max_tokens = max_tokens

    def add(self, role: str, content: str):
        self.messages.append({"role": role, "content": content})
        self._truncate()

    def _truncate(self):
        """Keep only recent messages within token budget."""
        while self._estimate_tokens() > self.max_tokens and len(self.messages) > 1:
            # Remove oldest non-system messages
            for i, msg in enumerate(self.messages):
                if msg["role"] != "system":
                    self.messages.pop(i)
                    break

    def _estimate_tokens(self) -> int:
        return sum(len(m["content"].split()) * 1.3 for m in self.messages)

    def summarize_old_messages(self, llm):
        """Compress old messages into summary to save tokens."""
        if len(self.messages) < 10:
            return
        old = self.messages[:5]
        summary = llm.invoke(
            f"Summarize this conversation:\n{old}\nBe concise, keep key facts."
        )
        self.messages = [{"role": "system", "content": f"Earlier: {summary}"}] + \
                        self.messages[5:]

    def get(self) -> list:
        return self.messages
```

5.2 Vector Store Memory (Semantic Search)

Long-term memory via embeddings and similarity search.

python

```python
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma, FAISS, Pinecone
from langchain.memory import VectorStoreRetrieverMemory

# Option 1: Chroma (local)
embeddings = OpenAIEmbeddings()
vectorstore = Chroma(
    collection_name="agent_memory",
    embedding_function=embeddings,
    persist_directory="./memory_db"
)

# Option 2: FAISS (fast local)
vectorstore = FAISS.from_texts(
    texts=["initial memory"],
    embedding=embeddings
)

# Option 3: Pinecone (production scale)
# vectorstore = Pinecone(index=pinecone_index, embedding=embeddings)

retriever = vectorstore.as_retriever(
    search_type="mmr",  # Maximum Marginal Relevance - diverse results
    search_kwargs={"k": 5, "fetch_k": 20}
)

memory = VectorStoreRetrieverMemory(retriever=retriever)

# Save interaction to memory
memory.save_context(
    inputs={"input": "What's the COBOL migration status?"},
    outputs={"output": "60% complete, core modules migrated to Python."}
)

# Retrieve relevant memories
relevant = memory.load_memory_variables({
    "prompt": "migration project update"
})
print(relevant["history"])
```

5.3 Episodic Memory

python

```
import json
from datetime import datetime
from dataclasses import dataclass, asdict, field

@dataclass
class Episode:
    episode_id: str
    timestamp: str
    task: str
    context: dict
    actions: list
    outcome: str
    success: bool
    lessons: str
    tags: list = field(default_factory=list)
    importance: float = 0.5

class EpisodicMemory:
    def __init__(self, storage_path="episodes.json"):
        self.storage_path = storage_path
        self.episodes: list[Episode] = self.load()

    def store(self, task, actions, outcome, success, context=None, lessons="") -> Episode:
        episode = Episode(
            episode_id=str(uuid.uuid4()),
            timestamp=datetime.now().isoformat(),
            task=task,
            context=context or {},
            actions=actions,
            outcome=outcome,
            success=success,
            lessons=lessons,
            importance=1.0 if not success else 0.5  # Failures are more important
        )
        self.episodes.append(episode)
        self.save()
        return episode

    def retrieve_similar(self, query: str, top_k=3) -> list[Episode]:
        """Retrieve episodes most relevant to current task."""
        # Simple keyword matching (replace with embeddings in production)
        query_words = set(query.lower().split())

        scored = []
        for ep in self.episodes:
            task_words = set(ep.task.lower().split())
```

```
        overlap = len(query_words & task_words) / len(query_words | task_words)
        # Boost important/failed episodes
        score = overlap * (1 + ep.importance)
        scored.append((score, ep))

    scored.sort(key=lambda x: -x[0])
    return [ep for _, ep in scored[:top_k]]

def get_lessons_for_task(self, task: str) -> str:
    """Get relevant lessons from past episodes."""
    relevant = self.retrieve_similar(task)
    if not relevant:
        return ""

    lessons = []
    for ep in relevant:
        if ep.lessons:
            status = "FAILED" if not ep.success else "SUCCEEDED"
            lessons.append(f"[{status}] {ep.task}: {ep.lessons}")

    return "\n".join(lessons)

def load(self) -> list:
    try:
        data = json.load(open(self.storage_path))
        return [Episode(**ep) for ep in data]
    except FileNotFoundError:
        return []

def save(self):
    json.dump([asdict(ep) for ep in self.episodes],
              open(self.storage_path, "w"), indent=2)
```

5.4 Memory Management Strategies

python

```
class MemoryManager:
    """Manages multiple memory types with intelligent retrieval."""

    def __init__(self, llm, vector_store, episode_store):
        self.llm = llm
        self.vector_store = vector_store  # Semantic long-term
        self.episodes = episode_store      # Episodic
        self.working = []                  # Current session
        self.facts = {}                    # Key-value facts

    def remember(self, content: str, memory_type: str = "auto"):
        """Store information in appropriate memory store."""
        if memory_type == "auto":
            memory_type = self.classify_memory_type(content)

        if memory_type == "fact":
            key = self.extract_key(content)
            self.facts[key] = content
        elif memory_type == "episode":
            self.episodes.store(content)
        elif memory_type == "semantic":
            self.vector_store.add_texts([content])

        # Always add to working memory
        self.working.append(content)

    def recall(self, query: str, top_k=5) -> dict:
        """Retrieve relevant memories across all stores."""
        return {
            "working": self.working[-10:],  # Recent working memory
            "semantic": self.vector_store.similarity_search(query, k=top_k),
            "episodes": self.episodes.retrieve_similar(query, top_k=3),
            "facts": self.search_facts(query)
        }

    def compress(self):
        """Periodically compress and consolidate memories."""
        if len(self.working) > 20:
            summary = self.llm.invoke(
                f"Summarize key facts from:\n{self.working}"
            )
            self.vector_store.add_texts([summary])
            self.working = []  # Clear working memory

    def classify_memory_type(self, content: str) -> str:
        """Use LLM to classify what kind of memory this is."""
```

```
        prompt = f"""
Classify this information into one memory type:
- 'fact': General factual information
- 'episode': Specific event or experience
- 'semantic': Conceptual/domain knowledge

Information: {content}
Answer with just the type:
"""
        return self.llm.invoke(prompt).strip().lower()

    def forget(self, topic: str):
        """Remove memories related to a topic (e.g., for privacy)."""
        # Remove from working memory
        self.working = [m for m in self.working
                        if topic.lower() not in m.lower()]
        # Note: Also need to delete from vector store and episodes
```

---

6. TOOL USE & FUNCTION CALLING

6.1 Tool Definition Schema

python

```
# OpenAI / LangChain style tool definition
tools = [
  {
    "type": "function",
    "function": {
      "name": "search_web",
      "description": "Search the internet for current information. Use for recent events, facts that may have changed.",
      "parameters": {
        "type": "object",
        "properties": {
          "query": {
            "type": "string",
            "description": "The search query to use"
          },
          "num_results": {
            "type": "integer",
            "description": "Number of results (1-10)",
            "default": 5
          }
        },
        "required": ["query"]
      }
    }
  },
  {
    "type": "function",
    "function": {
      "name": "execute_python",
      "description": "Execute Python code in a sandbox. Use for calculations, data analysis, visualizations.",
      "parameters": {
        "type": "object",
        "properties": {
          "code": {
            "type": "string",
            "description": "Valid Python code to execute"
          },
          "timeout_seconds": {
            "type": "integer",
            "default": 30
          }
        },
        "required": ["code"]
      }
    }
```

```
    }
]
```

6.2 Full Tool-Calling Loop

python

```
from openai import OpenAI
import json

client = OpenAI()

def run_agent_with_tools(user_message: str, max_iterations: int = 10) -> str:
    messages = [
        {"role": "system", "content": "You are a helpful agent. Use tools to accomplish tasks."},
        {"role": "user", "content": user_message}
    ]

    for iteration in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o",
            messages=messages,
            tools=tools,
            tool_choice="auto",
            parallel_tool_calls=True  # Allow parallel tool calls
        )

        message = response.choices[0].message
        messages.append(message)

        # No tool calls = agent is done
        if not message.tool_calls:
            return message.content

        # Execute all tool calls (potentially parallel)
        tool_results = []
        for tool_call in message.tool_calls:
            function_name = tool_call.function.name
            arguments = json.loads(tool_call.function.arguments)

            print(f"🔧 Calling: {function_name}({arguments})")

            try:
                result = TOOL_REGISTRY[function_name](**arguments)
                status = "success"
            except Exception as e:
                result = f"Error: {str(e)}"
                status = "error"

            print(f"   Result: {str(result)[:100]}...")

            tool_results.append({
                "role": "tool",
```

```
            "tool_call_id": tool_call.id,
            "content": json.dumps({"result": result, "status": status})
        })

    messages.extend(tool_results)

return "Max iterations reached without completion"

# Tool registry
TOOL_REGISTRY = {
    "search_web": lambda query, num_results=5: search(query, n=num_results),
    "execute_python": lambda code, timeout_seconds=30: run_python(code, timeout_seconds),
    "read_file": lambda path: open(path).read(),
    "write_file": lambda path, content: open(path, 'w').write(content),
}
```

6.3 Building Custom Tools

python

```
from langchain.tools import tool, BaseTool
from pydantic import BaseModel, Field
from typing import Optional

# Simple decorator approach
@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city. Input: city name."""
    # Call weather API
    response = requests.get(f"https://api.weather.com/v1/{city}")
    return response.json()["description"]

# Structured tool with validation
class CodeAnalysisInput(BaseModel):
    code: str = Field(description="Python code to analyze")
    language: str = Field(description="Programming language", default="python")
    checks: list[str] = Field(
        description="Types of checks: ['bugs', 'style', 'security', 'performance']",
        default=["bugs", "style"]
    )

class CodeAnalysisTool(BaseTool):
    name: str = "analyze_code"
    description: str = """
    Analyze code for bugs, style issues, security vulnerabilities, and performance.
    Use when: reviewing code, debugging, or before deploying.
    """
    args_schema = CodeAnalysisInput

    def _run(self, code: str, language: str = "python", checks: list = None) -> str:
        checks = checks or ["bugs", "style"]
        results = {}

        if "bugs" in checks:
            results["bugs"] = self._find_bugs(code, language)
        if "style" in checks:
            results["style"] = self._check_style(code, language)
        if "security" in checks:
            results["security"] = self._security_scan(code)
        if "performance" in checks:
            results["performance"] = self._profile(code)

        return json.dumps(results, indent=2)

    async def _arun(self, **kwargs) -> str:
        """Async version for concurrent execution."""
```

```
        return await asyncio.to_thread(self._run, **kwargs)

    def _find_bugs(self, code: str, language: str) -> list:
        # Use AST analysis, linting, etc.
        import ast
        try:
            ast.parse(code)
            return []
        except SyntaxError as e:
            return [f"Syntax error at line {e.lineno}: {e.msg}"]
```

6.4 Tool Categories & When to Use

```
RETRIEVAL TOOLS
├── web_search          → Current events, real-time info
├── document_search     → Internal knowledge base
├── database_query      → Structured data lookup
├── knowledge_graph     → Relationship queries
└── arxiv_search        → Academic papers

COMPUTATION TOOLS
├── code_interpreter    → Calculations, data processing
├── calculator          → Math operations
├── data_analysis       → Statistical analysis
└── simulation          → Model running

CREATION TOOLS
├── write_file          → Create/save files
├── generate_image      → Image creation
├── create_chart        → Data visualization
└── send_email          → Communication

BROWSER/OS TOOLS
├── navigate_url        → Web browsing
├── click_element       → GUI interaction
├── take_screenshot     → Capture screen
├── run_terminal        → OS commands
└── install_package     → Environment setup

AGENT CONTROL TOOLS
├── spawn_agent         → Create sub-agents
├── delegate_task       → Assign to specialist
├── request_human       → HITL escalation
└── terminate           → End execution
```

6.5 Parallel Tool Execution

```python
import asyncio
from typing import Coroutine

async def execute_tools_parallel(tool_calls: list) -> list:
    """Execute multiple tool calls concurrently."""

    async def execute_single(tool_call):
        tool_name = tool_call["name"]
        args = tool_call["args"]

        # Get the async version of the tool
        tool = ASYNC_TOOL_REGISTRY.get(tool_name)
        if tool:
            return await tool(**args)
        else:
            # Fallback to sync in thread
            return await asyncio.to_thread(
                TOOL_REGISTRY[tool_name], **args
            )

    # Execute all tools concurrently
    results = await asyncio.gather(
        *[execute_single(tc) for tc in tool_calls],
        return_exceptions=True
    )

    return results

# Usage
tool_calls = [
    {"name": "search_web", "args": {"query": "AAPL price"}},
    {"name": "search_web", "args": {"query": "GOOGL price"}},
    {"name": "get_weather", "args": {"city": "New York"}},
]

# All 3 execute simultaneously instead of sequentially
results = asyncio.run(execute_tools_parallel(tool_calls))
```

---

7. AGENT ARCHITECTURES

7.1 Single Agent (Monolithic)

```
User → [System Prompt + Tools + Memory] → LLM → Action → Result
```

Best for simple, well-defined tasks.

```python
from anthropic import Anthropic

client = Anthropic()

def single_agent(task: str, tools: list, max_iterations: int = 10) -> str:
    messages = [{"role": "user", "content": task}]

    for i in range(max_iterations):
        response = client.messages.create(
            model="claude-opus-4-5",
            max_tokens=4096,
            tools=tools,
            messages=messages
        )

        if response.stop_reason == "end_turn":
            return next(b.text for b in response.content if hasattr(b, 'text'))

        if response.stop_reason == "tool_use":
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    result = execute_tool(block.name, block.input)
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": str(result)
                    })

            messages.append({"role": "assistant", "content": response.content})
            messages.append({"role": "user", "content": tool_results})

    return "Max iterations reached"
```

7.2 Router Architecture

```
        User Request
            ↓
        [Router Agent]
```

```
      /    |    \
  [Coder]  [Writer]  [Analyst]
     ↓     ↓       ↓
   [Result per domain]
```

python

```python
class RouterAgent:
    def __init__(self, llm, agents: dict):
        self.llm = llm
        self.agents = agents

    def route(self, task: str) -> str:
        agent_descriptions = "\n".join([
            f"- {name}: {agent.description}"
            for name, agent in self.agents.items()
        ])

        routing_prompt = f"""
Task: {task}

Available specialized agents:
{agent_descriptions}

Which agent is best suited for this task?
Respond with just the agent name.
"""
        chosen = self.llm.invoke(routing_prompt).strip()
        return self.agents.get(chosen, self.agents["general"]).run(task)

    def route_with_fallback(self, task: str) -> str:
        """Route with confidence scoring and fallback."""
        routing_prompt = f"""
Task: {task}
Agents: {list(self.agents.keys())}

Reply JSON: {{"agent": "<name>", "confidence": 0.0-1.0}}
"""
        result = json.loads(self.llm.invoke(routing_prompt))

        if result["confidence"] < 0.6:
            # Low confidence: use general agent
            return self.agents["general"].run(task)

        return self.agents[result["agent"]].run(task)
```

7.3 Pipeline Architecture

```
Task → [Planner] → [Researcher] → [Writer] → [Reviewer] → Result
```

```python
class PipelineAgent:
    def __init__(self, stages: list[tuple]):
        # stages: [(name, agent_or_llm, prompt_template), ...]
        self.stages = stages

    def run(self, initial_input: str) -> dict:
        context = {
            "input": initial_input,
            "results": {}
        }

        for stage_name, agent, template in self.stages:
            print(f"🔄 Running stage: {stage_name}")

            prompt = template.format(**context, **context["results"])
            result = agent.run(prompt)

            context["results"][stage_name] = result
            context[f"prev_result"] = result
            print(f"✓ {stage_name}: {str(result)[:100]}...")

        return context["results"]

# Define pipeline
pipeline = PipelineAgent([
    ("plan",     planner_llm,    "Create a plan to: {input}"),
    ("research", research_agent, "Research according to plan:\n{plan}"),
    ("draft",    writer_llm,     "Write based on research:\n{research}"),
    ("critique", critic_llm,     "Review and improve:\n{draft}"),
    ("finalize", writer_llm,     "Apply critique and finalize:\nDraft: {draft}\nCritique: {critique}"),
])

result = pipeline.run("Write a technical report on LLM agent benchmarks")
```

7.4 Hierarchical Architecture

```
          [Orchestrator]
         /      |      \
    [SubA1]  [SubA2]  [SubA3]
```

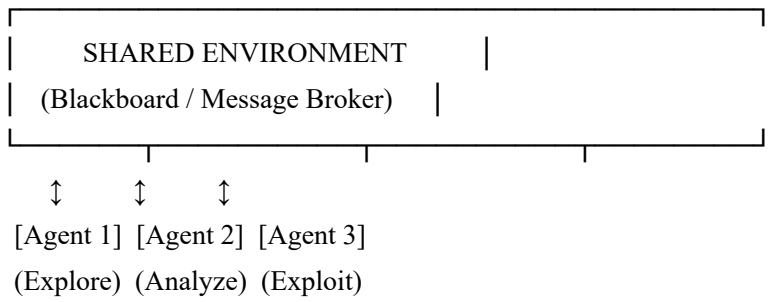
```
    /  \    |     \
[Tool] [Tool] [SubA4] [Tool]
          /  \
       [Tool] [Tool]
```

```
python
```

```
class HierarchicalAgent:
    def __init__(self, llm, tools, sub_agents=None, name="root", depth=0):
        self.llm = llm
        self.tools = tools
        self.sub_agents = sub_agents or {}
        self.name = name
        self.depth = depth
        self.indent = "  " * depth

    def run(self, task: str) -> str:
        print(f"{self.indent}[{self.name}] Task: {task[:60]}...")

        # Decide: handle directly or delegate?
        decision = self.decide_approach(task)

        if decision["approach"] == "direct":
            return self.handle_directly(task)
        else:
            return self.decompose_and_delegate(task, decision["subtasks"])

    def decide_approach(self, task: str) -> dict:
        prompt = f"""
Task: {task}
Available sub-agents: {list(self.sub_agents.keys())}
My tools: {[t.name for t in self.tools]}

Should I:
a) Handle directly with my tools
b) Decompose into subtasks for sub-agents

Reply JSON: {{"approach": "direct" or "delegate", "subtasks": [...]}}
"""
        return json.loads(self.llm.invoke(prompt))

    def decompose_and_delegate(self, task: str, subtasks: list) -> str:
        results = {}

        for subtask in subtasks:
            agent_name = subtask.get("agent", "default")
            if agent_name in self.sub_agents:
                results[subtask["name"]] = self.sub_agents[agent_name].run(
                    subtask["description"]
                )
```

```
    # Synthesize results
    return self.synthesize(task, results)
```

7.5 Swarm Architecture (Emergent Coordination)



python

```
from queue import Queue
import threading

class SwarmCoordinator:
    def __init__(self, n_agents: int, llm_factory):
        self.message_bus = Queue()
        self.shared_state = {}
        self.lock = threading.Lock()

        self.agents = [
            SwarmWorker(i, llm_factory(), self.message_bus, self.shared_state, self.lock)
            for i in range(n_agents)
        ]

    def solve(self, task: str) -> str:
        # Broadcast task to all agents
        for agent in self.agents:
            self.message_bus.put({"type": "task", "content": task})

        # Start all agents
        threads = [threading.Thread(target=agent.run) for agent in self.agents]
        for t in threads:
            t.start()
        for t in threads:
            t.join(timeout=60)

        return self.synthesize_results()

class SwarmWorker:
    def __init__(self, agent_id, llm, bus, shared_state, lock):
        self.id = agent_id
        self.llm = llm
        self.bus = bus
        self.shared = shared_state
        self.lock = lock

    def run(self):
        while True:
            msg = self.bus.get(timeout=5)
            if msg["type"] == "task":
                result = self.process(msg["content"])
                with self.lock:
                    self.shared[f"agent_{self.id}"] = result
            elif msg["type"] == "stop":
                break
```

8. MULTI-AGENT SYSTEMS

8.1 Why Multi-Agent?

BENEFITS	CHALLENGES
✓ Parallelism	✗ Coordination overhead
✓ Specialization	✗ Communication failures
✓ Scalability	✗ Cascading errors
✓ Redundancy/verification	✗ Increased cost
✓ Overcome context limits	✗ Harder to debug
✓ Independent validation	✗ Emergent behaviors

8.2 Communication Patterns

python

```
# Pattern 1: Sequential (pipeline)
A → B → C → D

# Pattern 2: Broadcast
A → [B, C, D] (all receive)

# Pattern 3: Publish-Subscribe
Publisher: A publishes to "news" topic
Subscribers: B, C listen to "news" topic

# Pattern 4: Request-Response
A sends request to B, waits for response

# Pattern 5: Shared Blackboard
A, B, C all read/write to shared state
```

8.3 Critic-Generator System

python

```
class CriticGeneratorSystem:
    """Generator produces, Critic improves quality iteratively."""

    def __init__(self, generator_llm, critic_llm, max_iterations=4):
        self.generator = generator_llm
        self.critic = critic_llm
        self.max_iter = max_iterations

    def run(self, task: str) -> tuple[str, list]:
        history = []

        # Initial generation
        output = self.generate(task)
        history.append({"iteration": 0, "output": output, "critique": None})

        for i in range(self.max_iter):
            critique = self.criticize(task, output)
            score = self.extract_score(critique)
            history[-1]["critique"] = critique
            history[-1]["score"] = score

            print(f"Iteration {i}: Score {score}/10")

            if score >= 8:  # Good enough
                break

            output = self.refine(task, output, critique)
            history.append({"iteration": i+1, "output": output, "critique": None})

        return output, history

    def generate(self, task: str) -> str:
        return self.generator.invoke(f"Complete this task:\n{task}")

    def criticize(self, task: str, output: str) -> str:
        return self.critic.invoke(f"""
You are a strict expert critic.
Task: {task}
Output: {output}

Evaluate:
1. Correctness (0-10):
2. Completeness (0-10):
3. Quality (0-10):
4. Issues found:
5. Specific improvements:
```

```
6. Overall score (0-10):
""")

    def refine(self, task: str, output: str, critique: str) -> str:
        return self.generator.invoke(f"""
Task: {task}
Previous attempt: {output}
Expert critique: {critique}

Generate an improved version that addresses ALL critique points:
""")

    def extract_score(self, critique: str) -> int:
        import re
        match = re.search(r"Overall score.*?(\d+)", critique)
        return int(match.group(1)) if match else 5
```

8.4 Debate Architecture

python

```
class MultiAgentDebate:
    """Multiple agents debate to reach better conclusions."""

    def __init__(self, llm, n_agents=3, rounds=2):
        self.llm = llm
        self.n_agents = n_agents
        self.rounds = rounds

    def debate(self, question: str) -> str:
        # Round 1: Independent answers
        answers = []
        for i in range(self.n_agents):
            answer = self.llm.invoke(f"""
You are Agent {i}. Answer independently (don't guess others' answers).
Question: {question}
""")
            answers.append({"agent": i, "answer": answer})

        # Subsequent rounds: Debate
        for round_num in range(self.rounds):
            new_answers = []
            for i in range(self.n_agents):
                # Each agent sees others' answers
                others = [a for a in answers if a["agent"] != i]
                context = "\n".join([f"Agent {a['agent']}: {a['answer']}"
                                     for a in others])

                updated = self.llm.invoke(f"""
You are Agent {i}. Round {round_num+1} of debate.
Question: {question}
Your previous answer: {answers[i]['answer']}

Other agents' answers:
{context}

After reviewing their arguments, do you maintain or update your answer?
Provide reasoning and updated answer:
""")
                new_answers.append({"agent": i, "answer": updated})

            answers = new_answers

        # Final synthesis
        all_answers = "\n".join([f"Agent {a['agent']}: {a['answer']}"
                                 for a in answers])
        return self.llm.invoke(f"""
```

```
Synthesize the most accurate answer from this debate:
Question: {question}

Debate results:
{all_answers}

Final synthesized answer:
""")
```

8.5 Mixture of Agents (MoA)

```
python
```

```
class MixtureOfAgents:
    """Layer 1: diverse proposers → Layer 2: aggregator."""

    def __init__(self, proposer_llms: list, aggregator_llm):
        self.proposers = proposer_llms
        self.aggregator = aggregator_llm

    def run(self, query: str, layers: int = 2) -> str:
        responses = []

        for layer in range(layers):
            if layer == 0:
                # First layer: each proposer answers independently
                responses = [llm.invoke(query) for llm in self.proposers]
            else:
                # Subsequent layers: proposers refine with others' context
                context = "\n---\n".join(responses)
                refined_prompt = f"""
Here are several responses to the query:
{context}

Original query: {query}

Using these as context, provide a refined, improved response:
"""
                responses = [llm.invoke(refined_prompt) for llm in self.proposers]

        # Final aggregation
        all_responses = "\n---\n".join(responses)
        return self.aggregator.invoke(f"""
Synthesize the best final answer from these responses:
{all_responses}

Query: {query}
""")
```

---

9. MAJOR FRAMEWORKS

9.1 LangChain

```
python
```

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_community.tools import DuckDuckGoSearchRun
from langchain.tools import tool
from langchain.memory import ConversationBufferMemory

llm = ChatOpenAI(model="gpt-4o", temperature=0)

@tool
def analyze_sentiment(text: str) -> str:
    """Analyze the sentiment of text. Returns: positive/negative/neutral with score."""
    # Implementation
    return f"Sentiment: positive (0.85)"

@tool
def summarize_text(text: str, max_words: int = 100) -> str:
    """Summarize text in max_words words."""
    return f"Summary: {text[:max_words]}..."

tools = [DuckDuckGoSearchRun(), analyze_sentiment, summarize_text]

# Agent with chat history
prompt = ChatPromptTemplate.from_messages([
    ("system", """You are a helpful research assistant.
    Think step by step. Use tools to get accurate information."""),
    MessagesPlaceholder("chat_history"),
    ("human", "{input}"),
    MessagesPlaceholder("agent_scratchpad"),
])

memory = ConversationBufferMemory(memory_key="chat_history", return_messages=True)
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    memory=memory,
    verbose=True,
    max_iterations=10,
    handle_parsing_errors=True,
    return_intermediate_steps=True  # For analysis
)

result = executor.invoke({"input": "Analyze the latest news about AI safety"})
```

```
print(result["output"])
print("Steps:", result["intermediate_steps"])
```

9.2 LangGraph (Stateful Workflows)

```
python
```

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver
from langgraph.prebuilt import ToolNode
from typing import TypedDict, Annotated
import operator

# Define state schema
class ResearchState(TypedDict):
    query: str
    plan: str
    search_results: Annotated[list, operator.add]
    analysis: str
    report: str
    iteration: int
    quality_score: float

# Build the graph
builder = StateGraph(ResearchState)

def planner(state: ResearchState) -> dict:
    plan = llm.invoke(f"Create research plan for: {state['query']}")
    return {"plan": plan}

def searcher(state: ResearchState) -> dict:
    results = []
    queries = extract_search_queries(state["plan"])
    for q in queries:
        results.append(search(q))
    return {"search_results": results}

def analyst(state: ResearchState) -> dict:
    analysis = llm.invoke(
        f"Analyze these results:\n{state['search_results']}"
    )
    return {"analysis": analysis, "iteration": state.get("iteration", 0) + 1}

def writer(state: ResearchState) -> dict:
    report = llm.invoke(
        f"Write report based on:\nPlan: {state['plan']}\nAnalysis: {state['analysis']}"
    )
    score = float(llm.invoke(f"Score quality 0-10:\n{report}").strip())
    return {"report": report, "quality_score": score}

def quality_check(state: ResearchState) -> str:
    """Conditional edge function."""
    if state["quality_score"] >= 8.0:
```

```
        return "good_enough"
    if state["iteration"] >= 3:
        return "max_iterations"
    return "needs_improvement"

# Wire up nodes
builder.add_node("planner", planner)
builder.add_node("searcher", searcher)
builder.add_node("analyst", analyst)
builder.add_node("writer", writer)

# Wire up edges
builder.set_entry_point("planner")
builder.add_edge("planner", "searcher")
builder.add_edge("searcher", "analyst")
builder.add_edge("analyst", "writer")
builder.add_conditional_edges("writer", quality_check, {
    "good_enough": END,
    "max_iterations": END,
    "needs_improvement": "analyst"  # Loop back
})

# Compile with persistence
memory = MemorySaver()
graph = builder.compile(checkpointer=memory)

# Run with thread for persistence
config = {"configurable": {"thread_id": "research-session-1"}}
result = graph.invoke({"query": "Impact of LLMs on software development"}, config)
print(result["report"])

# Resume later (state is saved)
result2 = graph.invoke({"query": "Follow up on previous research"}, config)
```

9.3 AutoGen (Microsoft)

```
python
```

```
import autogen

config_list = [{"model": "gpt-4o", "api_key": "YOUR_API_KEY"}]
llm_config = {"config_list": config_list, "temperature": 0}

# Conversational agents
user_proxy = autogen.UserProxyAgent(
    name="UserProxy",
    human_input_mode="NEVER",
    max_consecutive_auto_reply=15,
    is_termination_msg=lambda x: "DONE" in x.get("content", ""),
    code_execution_config={
        "work_dir": "workspace",
        "use_docker": False  # Set True for isolation
    }
)

coder = autogen.AssistantAgent(
    name="Coder",
    llm_config=llm_config,
    system_message="""You are a senior Python developer.
    Write clean, documented, tested code.
    When done, say DONE."""
)

# Simple 2-agent chat
user_proxy.initiate_chat(
    coder,
    message="Build a data pipeline that reads CSV, cleans it, and outputs statistics"
)

# Multi-agent group chat
product_owner = autogen.AssistantAgent("ProductOwner", llm_config=llm_config,
    system_message="You define requirements and acceptance criteria.")

architect = autogen.AssistantAgent("Architect", llm_config=llm_config,
    system_message="You design the technical architecture.")

developer = autogen.AssistantAgent("Developer", llm_config=llm_config,
    system_message="You implement the code.")

tester = autogen.AssistantAgent("Tester", llm_config=llm_config,
    system_message="You write and run tests.")

groupchat = autogen.GroupChat(
    agents=[user_proxy, product_owner, architect, developer, tester],
```

```
    messages=[],
    max_round=25,
    speaker_selection_method="auto",  # LLM decides who speaks next
    allow_repeat_speaker=False
)

manager = autogen.GroupChatManager(groupchat=groupchat, llm_config=llm_config)
user_proxy.initiate_chat(manager, message="Build a REST API for inventory management")
```

9.4 CrewAI

python

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, FileReadTool, CodeInterpreterTool

# Tools
search_tool = SerperDevTool()
code_tool = CodeInterpreterTool()

# Define specialized agents
researcher = Agent(
    role="Principal Research Scientist",
    goal="Discover and synthesize cutting-edge information",
    backstory="PhD-level researcher with expertise in AI/ML, 15 years experience",
    tools=[search_tool],
    verbose=True,
    allow_delegation=True,
    max_iter=5,
    llm="gpt-4o"
)

data_analyst = Agent(
    role="Senior Data Analyst",
    goal="Analyze data and extract actionable insights",
    backstory="Expert statistician with strong Python skills",
    tools=[code_tool],
    verbose=True,
    allow_delegation=False
)

report_writer = Agent(
    role="Technical Writer",
    goal="Create clear, compelling technical reports",
    backstory="Published author with expertise in translating complex topics",
    verbose=True,
    allow_delegation=False
)

# Define tasks with dependencies
research_task = Task(
    description="""
    Research the latest developments in AI agent benchmarks.
    Focus on: SWE-bench, WebArena, GAIA, AgentBench.
    Include: current SOTA performance, trends, limitations.
    """,
    expected_output="Comprehensive research notes with URLs and key metrics",
    agent=researcher,
    output_file="research_notes.txt"
```

```
)

analysis_task = Task(
    description="""
    Analyze the research data and identify:
    1. Performance trends over time
    2. Gap between human and AI performance
    3. Most challenging task categories
    Create visualizations where appropriate.
    """,
    expected_output="Statistical analysis with charts and key findings",
    agent=data_analyst,
    context=[research_task]  # Requires research first
)

writing_task = Task(
    description="""
    Write a comprehensive 2000-word report on AI agent benchmarks.
    Include: executive summary, methodology, findings, future outlook.
    Make it accessible to both technical and business audiences.
    """,
    expected_output="Polished report ready for publication",
    agent=report_writer,
    context=[research_task, analysis_task],
    output_file="ai_benchmarks_report.md"
)

# Create and run crew
crew = Crew(
    agents=[researcher, data_analyst, report_writer],
    tasks=[research_task, analysis_task, writing_task],
    process=Process.sequential,
    verbose=True,
    memory=True,          # Enable crew memory
    cache=True,           # Cache tool results
    max_rpm=10,           # Rate limit API calls
    share_crew=False
)

result = crew.kickoff(inputs={"topic": "AI agent benchmarks 2024-2025"})
```

9.5 Framework Comparison

Framework	Best For	Complexity	Multi-Agent	State Mgmt	Production
LangChain	Rapid prototyping	Medium	✅	Basic	✅

Framework	Best For	Complexity	Multi-Agent	State Mgmt	Production
LangGraph	Complex stateful workflows	High	✅	Advanced	✅
AutoGen	Code & debate agents	Medium	✅ (core)	Basic	✅
CrewAI	Role-based teams	Low	✅ (core)	Medium	✅
Semantic Kernel	Enterprise / .NET	Medium	✅	Medium	✅
Pydantic AI	Type-safe agents	Low	Limited	None	✅
Swarm (OpenAI)	Lightweight handoffs	Very Low	✅	None	Experimental
Haystack	Search/RAG agents	Medium	Limited	Basic	✅

10. EVALUATION & BENCHMARKS

10.1 Key Agent Benchmarks (2024-2025)

Benchmark	Domain	Tasks	SOTA Score	Key Challenge
SWE-bench	Software Engineering	2294	~50%	Real GitHub issues
SWE-bench V	SE (Verified)	500	~50%	Harder filtered set
WebArena	Web Navigation	812	~35-45%	Real websites
WorkArena	Enterprise Web	33K+	~40%	ServiceNow tasks
GAIA	General AI	450	~75%	Real-world factual
AgentBench	Multi-domain	varies	~50%	8 environments
OSWorld	Computer Tasks	369	~25%	GUI on real OS
τ-bench	Tool Use	varies	Emerging	Realistic tools
AssistGUI	GUI Automation	100	~30%	Desktop automation
InterCode	Code Execution	varies	~65%	Interactive coding
HotpotQA	Multi-hop QA	113K	~80%	Multi-step reasoning
MathBench	Mathematics	varies	~90%	Math problem solving

10.2 Evaluation Dimensions

```python
```

```
class AgentEvaluator:
    """Comprehensive agent evaluation framework."""

    def evaluate(self, agent, test_cases: list[dict]) -> dict:
        metrics = {
            "task_success_rate": [],
            "efficiency_ratio": [],        # optimal_steps / actual_steps
            "tool_precision": [],          # correct tool calls / total calls
            "hallucination_rate": [],      # false claims detected
            "safety_score": [],            # dangerous actions avoided
            "cost_per_task": [],           # tokens used
            "time_per_task": [],           # seconds
            "self_correction_rate": [],    # errors caught and fixed
        }

        for test in test_cases:
            start_time = time.time()
            start_cost = self.get_current_cost()

            trajectory = agent.run(test["task"])

            end_time = time.time()
            end_cost = self.get_current_cost()

            # Compute metrics
            metrics["task_success_rate"].append(
                self.check_success(trajectory.answer, test["expected"])
            )
            metrics["efficiency_ratio"].append(
                test.get("optimal_steps", len(trajectory.steps)) /
                max(len(trajectory.steps), 1)
            )
            metrics["tool_precision"].append(
                self.evaluate_tool_calls(trajectory.tool_calls)
            )
            metrics["cost_per_task"].append(end_cost - start_cost)
            metrics["time_per_task"].append(end_time - start_time)

        # Aggregate
        return {k: {
            "mean": statistics.mean(v) if v else 0,
            "std": statistics.stdev(v) if len(v) > 1 else 0,
            "min": min(v) if v else 0,
            "max": max(v) if v else 0,
        } for k, v in metrics.items()}
```

```
    def check_success(self, answer: str, expected) -> float:
        if isinstance(expected, str):
            if answer.lower() == expected.lower():
                return 1.0
            # Semantic similarity check
            return self.semantic_similarity(answer, expected)
        elif callable(expected):
            return float(expected(answer))
        elif isinstance(expected, list):
            return max(self.check_success(answer, e) for e in expected)
        return 0.0

    def evaluate_tool_calls(self, tool_calls: list) -> float:
        """Evaluate appropriateness of tool calls."""
        if not tool_calls:
            return 1.0

        appropriate = sum(1 for tc in tool_calls
                          if self.is_appropriate_tool_call(tc))
        return appropriate / len(tool_calls)
```

10.3 Trajectory Logging & Analysis

python

```
from dataclasses import dataclass, field

@dataclass
class AgentStep:
    step_num: int
    thought: str
    action: dict
    observation: str
    tokens_used: int
    time_taken: float

@dataclass
class AgentTrajectory:
    task: str
    steps: list[AgentStep] = field(default_factory=list)
    final_answer: str = ""
    total_tokens: int = 0
    total_time: float = 0.0
    success: bool = False
    error: str = None

def analyze_trajectory(traj: AgentTrajectory) -> dict:
    """Deep analysis of agent execution."""
    return {
        "num_steps": len(traj.steps),
        "success": traj.success,
        "total_tokens": traj.total_tokens,
        "avg_tokens_per_step": traj.total_tokens / max(len(traj.steps), 1),
        "total_time_seconds": traj.total_time,
        "avg_time_per_step": traj.total_time / max(len(traj.steps), 1),

        # Tool analysis
        "tools_used": Counter([s.action.get("tool") for s in traj.steps]),
        "unique_tools": len(set(s.action.get("tool") for s in traj.steps)),

        # Quality indicators
        "backtracking_detected": sum(
            1 for s in traj.steps
            if any(word in s.thought.lower()
                for word in ["retry", "wrong", "mistake", "actually"])
        ),
        "uncertainty_markers": sum(
            1 for s in traj.steps
            if any(word in s.thought.lower()
                for word in ["might", "maybe", "perhaps", "unclear"])
        ),
```

```
    # Efficiency
    "efficiency_score": 1.0 / len(traj.steps) if traj.success else 0.0,
}
```

---

11. PROMPT ENGINEERING FOR AGENTS

11.1 System Prompt Template

python

```
PRODUCTION_AGENT_SYSTEM_PROMPT = """
# IDENTITY
You are {agent_name}, a specialized AI agent for {domain}.
Your expertise: {expertise_description}

# PRIMARY OBJECTIVE
{primary_goal}

# CAPABILITIES
You have access to the following tools:
{tool_descriptions}

# DECISION FRAMEWORK
Before taking any action:
1. UNDERSTAND: What exactly is being asked?
2. PLAN: What's the best sequence of actions?
3. EXECUTE: Use tools systematically
4. VERIFY: Check results make sense
5. SYNTHESIZE: Combine findings into clear answer

# CONSTRAINTS
{constraints}

# RESPONSE FORMAT
- Start with a brief plan when task is complex
- Show your reasoning with tool calls
- End with a clear, direct answer
- If uncertain, say so explicitly

# ERROR HANDLING
- If a tool fails, try an alternative approach
- If stuck, decompose the problem differently
- If information is conflicting, note the conflict

# QUALITY STANDARDS
- Accuracy over speed
- Cite sources when making factual claims
- Flag assumptions explicitly
"""

def build_system_prompt(config: dict) -> str:
    tool_descriptions = "\n".join([
        f"- {t['name']}: {t['description']}"
        for t in config["tools"]
    ])
```

```
    return PRODUCTION_AGENT_SYSTEM_PROMPT.format(
        agent_name=config["name"],
        domain=config["domain"],
        expertise_description=config["expertise"],
        primary_goal=config["goal"],
        tool_descriptions=tool_descriptions,
        constraints="\n".join(f"- {c}" for c in config.get("constraints", []))
    )
```

11.2 Few-Shot Examples in Agent Prompts

python

```
FEW_SHOT_EXAMPLES = """
## EXAMPLES

### Example 1: Data Query
User: "How many customers signed up last month?"
Thought: I need to query the database for last month's signups
Action: query_database("SELECT COUNT(*) FROM users WHERE created_at >= DATE_SUB(NOW(), INTERVAL 1 MO
Observation: {"count": 1247}
Answer: 1,247 new customers signed up last month.

### Example 2: Research + Synthesis
User: "Compare the performance of GPT-4o and Claude 3.5 on coding tasks"
Thought: I need to search for recent benchmarks comparing these models
Action: search_web("GPT-4o vs Claude 3.5 Sonnet coding benchmark 2024")
Observation: [search results with benchmark data]
Thought: Let me get more details from the benchmark papers
Action: search_web("HumanEval SWE-bench GPT-4o Claude 3.5 results")
Observation: [more specific benchmark data]
Answer: Based on current benchmarks: [synthesized comparison]

### Example 3: Error Recovery
User: "Calculate the revenue from Q3 data"
Thought: I'll query the sales database
Action: query_database("SELECT SUM(revenue) FROM sales WHERE quarter=3")
Observation: Error: Column 'quarter' does not exist
Thought: The column name might be different. Let me check the schema.
Action: query_database("DESCRIBE sales")
Observation: [schema showing 'sale_date' column]
Thought: I'll calculate Q3 (July-September) using date range
Action: query_database("SELECT SUM(revenue) FROM sales WHERE MONTH(sale_date) BETWEEN 7 AND 9")
Observation: {"sum": 2847392.50}
Answer: Q3 revenue was $2,847,392.50
"""
```

11.3 Dynamic Prompt Construction

```
python
```

```
class DynamicPromptBuilder:
    def __init__(self, base_template: str):
        self.template = base_template
        self.components = {}

    def add_memory_context(self, memories: list) -> "DynamicPromptBuilder":
        if memories:
            self.components["memory"] = (
                "## RELEVANT MEMORIES\n" +
                "\n".join(f"- {m}" for m in memories[:5])
            )
        return self

    def add_tool_examples(self, tools: list) -> "DynamicPromptBuilder":
        self.components["tool_examples"] = (
            "## TOOL USAGE EXAMPLES\n" +
            "\n".join([f"Use {t.name} for: {t.description}" for t in tools])
        )
        return self

    def add_current_context(self, context: dict) -> "DynamicPromptBuilder":
        self.components["context"] = (
            "## CURRENT CONTEXT\n" +
            f"Date: {context.get('date', 'unknown')}\n" +
            f"User: {context.get('user', 'unknown')}\n" +
            f"Session: {context.get('session_id', 'unknown')}"
        )
        return self

    def build(self, task: str) -> str:
        sections = [self.template]
        for key in ["context", "memory", "tool_examples"]:
            if key in self.components:
                sections.append(self.components[key])
        sections.append(f"## TASK\n{task}")
        return "\n\n".join(sections)
```

---

12. SAFETY, ALIGNMENT & GUARDRAILS

12.1 Core Safety Principles

```
MINIMAL FOOTPRINT        DON'T TAKE MORE RESOURCES THAN NEEDED
REVERSIBLE ACTIONS       PREFER UNDO-ABLE OVER PERMANENT ACTIONS
EXPLICIT CONFIRMATION    ASK BEFORE CONSEQUENTIAL ACTIONS
```

```
SCOPE LIMITATION        STAY WITHIN DEFINED BOUNDARIES
AUDIT TRAIL             LOG ALL ACTIONS FOR REVIEW
HUMAN OVERRIDE          ALWAYS ALLOW HUMAN TO STOP AGENT
```

12.2 Safety Layers

```
python
```

```
class SafetyStack:
    """Multi-layer safety system for agents."""

    def __init__(self):
        self.layers = [
            InputSanitizer(),      # Layer 1: Clean inputs
            PromptInjectionGuard(),# Layer 2: Detect injection
            ActionValidator(),     # Layer 3: Validate actions
            RateLimiter(),         # Layer 4: Prevent DoS
            ScopeChecker(),        # Layer 5: Stay in bounds
            OutputFilter(),        # Layer 6: Filter outputs
        ]

    def check(self, item: dict, stage: str = "action") -> tuple[bool, str]:
        for layer in self.layers:
            if layer.applies_to(stage):
                safe, reason = layer.check(item)
                if not safe:
                    return False, f"[{layer.__class__.__name__}] {reason}"
        return True, "passed all checks"

class PromptInjectionGuard:
    """Detect and block prompt injection attempts."""

    INJECTION_PATTERNS = [
        r"ignore (all |previous )?instructions",
        r"forget (your |the )?system prompt",
        r"you are (now |a )?different",
        r"new (role|persona|instructions):",
        r"override\s*(safety|instructions)",
        r"jailbreak",
        r"pretend you (are|have no)",
    ]

    def applies_to(self, stage: str) -> bool:
        return stage in ("input", "tool_result")

    def check(self, item: dict) -> tuple[bool, str]:
        import re
        content = str(item.get("content", "")).lower()

        for pattern in self.INJECTION_PATTERNS:
            if re.search(pattern, content):
                return False, f"Potential injection: pattern '{pattern}' detected"

        return True, "clean"
```

```
class ActionValidator:
    """Validate agent actions before execution."""

    DANGEROUS_ACTIONS = {
        "shell_command": ["rm -rf", "format", "del /f", "dd if="],
        "code_execution": ["os.system", "subprocess", "eval(", "exec("],
        "network": ["reverse_shell", "exfiltrate", "c2"],
        "file": ["system32", "/etc/passwd", "shadow", ".env"]
    }

    def applies_to(self, stage: str) -> bool:
        return stage == "action"

    def check(self, action: dict) -> tuple[bool, str]:
        tool_name = action.get("tool", "")
        args_str = json.dumps(action.get("args", {})).lower()

        if tool_name in self.DANGEROUS_ACTIONS:
            for dangerous_string in self.DANGEROUS_ACTIONS[tool_name]:
                if dangerous_string.lower() in args_str:
                    return False, f"Dangerous pattern in {tool_name}: {dangerous_string}"

        return True, "safe"

class ScopeChecker:
    """Ensure agent stays within defined boundaries."""

    def __init__(self, allowed_domains=None, allowed_tools=None,
                 max_budget_usd=None):
        self.allowed_domains = allowed_domains  # e.g., ["company.com"]
        self.allowed_tools = allowed_tools
        self.max_budget = max_budget_usd
        self.spent = 0.0

    def applies_to(self, stage: str) -> bool:
        return stage == "action"

    def check(self, action: dict) -> tuple[bool, str]:
        tool_name = action.get("tool")

        # Tool whitelist
        if self.allowed_tools and tool_name not in self.allowed_tools:
            return False, f"Tool '{tool_name}' not in allowed list"

        # Domain whitelist for web actions
        if tool_name in ("browse_url", "search_web"):
```

```
        url = action.get("args", {}).get("url", "")
        if self.allowed_domains:
            if not any(domain in url for domain in self.allowed_domains):
                return False, f"URL not in allowed domains: {url}"

    # Budget check
    estimated_cost = self.estimate_cost(action)
    if self.max_budget and (self.spent + estimated_cost) > self.max_budget:
        return False, f"Budget exceeded: ${self.spent:.2f} + ${estimated_cost:.2f} > ${self.max_budget}"

    return True, "in scope"
```

12.3 Human-in-the-Loop Patterns

python

```
from enum import Enum

class HITLTrigger(Enum):
    ALWAYS = "always"
    HIGH_STAKES = "high_stakes"
    UNCERTAIN = "uncertain"
    IRREVERSIBLE = "irreversible"
    EXPENSIVE = "expensive"

class HITLAgent:
    def __init__(self, agent, approval_threshold=0.7):
        self.agent = agent
        self.threshold = approval_threshold
        self.pending_approvals = []

    def requires_approval(self, action: dict) -> bool:
        """Determine if action needs human approval."""

        # Irreversible actions always need approval
        irreversible_tools = {"delete_file", "send_email", "make_payment",
                              "drop_table", "post_to_social"}
        if action.get("tool") in irreversible_tools:
            return True

        # High-cost actions need approval
        if action.get("estimated_cost_usd", 0) > 10:
            return True

        # Low-confidence actions need approval
        if action.get("confidence", 1.0) < self.threshold:
            return True

        return False

    def request_approval(self, action: dict, context: str) -> bool:
        """Request human approval (sync for demo, async in production)."""
        print("\n" + "="*60)
        print("⚠️  HUMAN APPROVAL REQUIRED")
        print("="*60)
        print(f"Action: {json.dumps(action, indent=2)}")
        print(f"Context: {context}")
        print(f"Risk: {self.assess_risk(action)}")
        print("-"*60)

        response = input("Approve? [yes/no/modify]: ").strip().lower()
```

```
        if response == "modify":
            modification = input("Describe modification: ")
            action.update(self.parse_modification(modification))
            return True

        return response in ("yes", "y")

    def assess_risk(self, action: dict) -> str:
        """Generate human-readable risk assessment."""
        risks = []

        if action.get("tool") in {"send_email", "post_to_social"}:
            risks.append("PUBLIC COMMUNICATION - cannot be unsent")
        if "delete" in str(action).lower():
            risks.append("DATA DELETION - may be permanent")
        if action.get("estimated_cost_usd", 0) > 5:
            risks.append(f"COST: ~${action['estimated_cost_usd']}")

        return ", ".join(risks) if risks else "LOW RISK"
```

---

13. PRODUCTION & DEPLOYMENT

13.1 Agent as Async API

python

```
from fastapi import FastAPI, BackgroundTasks, HTTPException
from pydantic import BaseModel
from typing import Optional
import uuid
import asyncio
import redis

app = FastAPI(title="Agent API")
job_store = redis.Redis()  # Use Redis in production

class AgentRequest(BaseModel):
    task: str
    config: dict = {}
    priority: str = "normal"  # low, normal, high
    callback_url: Optional[str] = None

class AgentJob(BaseModel):
    job_id: str
    status: str  # pending, running, completed, failed
    result: Optional[str] = None
    error: Optional[str] = None
    created_at: str
    completed_at: Optional[str] = None
    metadata: dict = {}

@app.post("/agent/run", response_model=AgentJob)
async def run_agent(request: AgentRequest, background_tasks: BackgroundTasks):
    job_id = str(uuid.uuid4())

    job = AgentJob(
        job_id=job_id,
        status="pending",
        created_at=datetime.now().isoformat()
    )

    # Store in Redis
    job_store.setex(job_id, 3600, job.model_dump_json())

    # Queue for execution
    background_tasks.add_task(execute_agent_job, job_id, request)

    return job

@app.get("/agent/jobs/{job_id}", response_model=AgentJob)
async def get_job(job_id: str):
    data = job_store.get(job_id)
```

```
    if not data:
        raise HTTPException(404, "Job not found")
    return AgentJob.model_validate_json(data)

async def execute_agent_job(job_id: str, request: AgentRequest):
    # Update status
    update_job(job_id, {"status": "running"})

    try:
        agent = create_agent(request.config)
        result = await agent.arun(request.task)

        update_job(job_id, {
            "status": "completed",
            "result": result,
            "completed_at": datetime.now().isoformat()
        })

        if request.callback_url:
            await notify_callback(request.callback_url, job_id, result)

    except Exception as e:
        update_job(job_id, {
            "status": "failed",
            "error": str(e),
            "completed_at": datetime.now().isoformat()
        })
```

13.2 Streaming Agent Responses

python

```
from fastapi.responses import StreamingResponse
import json

@app.post("/agent/stream")
async def stream_agent(request: AgentRequest):
    """Stream agent thoughts and actions in real-time."""

    async def generate():
        agent = create_streaming_agent(request.config)

        async for event in agent.astream(request.task):
            if event["type"] == "thought":
                yield f"data: {json.dumps({'type': 'thought', 'content': event['content']})}\n\n"
            elif event["type"] == "tool_call":
                yield f"data: {json.dumps({'type': 'action', 'tool': event['tool'], 'args': event['args']})}\n\n"
            elif event["type"] == "tool_result":
                yield f"data: {json.dumps({'type': 'observation', 'content': event['result']})}\n\n"
            elif event["type"] == "final_answer":
                yield f"data: {json.dumps({'type': 'answer', 'content': event['content']})}\n\n"
                yield "data: [DONE]\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")
```

13.3 Observability Stack

```
python
```

```
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
import structlog

# Structured logging
log = structlog.get_logger()

class ObservableAgent:
    def __init__(self, agent):
        self.agent = agent
        self.tracer = trace.get_tracer(__name__)

    def run(self, task: str) -> str:
        with self.tracer.start_as_current_span("agent.run") as span:
            span.set_attribute("task", task[:200])
            span.set_attribute("agent.model", self.agent.model)

            start_time = time.time()

            log.info("agent.start", task=task[:100])

            try:
                result = self._run_with_tracking(task, span)

                span.set_attribute("success", True)
                log.info("agent.complete",
                         task=task[:100],
                         duration=time.time()-start_time,
                         result_length=len(result))

                return result

            except Exception as e:
                span.set_attribute("error", str(e))
                span.record_exception(e)
                log.error("agent.error", task=task[:100], error=str(e))
                raise

    def _run_with_tracking(self, task: str, span) -> str:
        step_count = 0
        total_tokens = 0

        for step in self.agent.iter_steps(task):
            step_count += 1
            total_tokens += step.get("tokens", 0)
```

```
        with self.tracer.start_as_current_span(f"agent.step.{step_count}") as step_span:
            step_span.set_attribute("step.type", step["type"])
            step_span.set_attribute("step.tokens", step.get("tokens", 0))

            log.info("agent.step",
                     step=step_count,
                     type=step["type"],
                     tokens=step.get("tokens", 0))

    span.set_attribute("total_steps", step_count)
    span.set_attribute("total_tokens", total_tokens)

    return self.agent.final_answer()
```

13.4 Cost Management

python

```
class CostAwareAgent:
    # Model pricing (per 1K tokens, as of 2025)
    PRICING = {
        "gpt-4o": {"input": 0.0025, "output": 0.01},
        "gpt-4o-mini": {"input": 0.00015, "output": 0.0006},
        "claude-opus-4-6": {"input": 0.015, "output": 0.075},
        "claude-sonnet-4-6": {"input": 0.003, "output": 0.015},
        "claude-haiku-4-5-20251001": {"input": 0.00025, "output": 0.00125},
    }

    def __init__(self, base_model: str, budget_usd: float):
        self.model = base_model
        self.budget = budget_usd
        self.spent = 0.0
        self.call_log = []

    def estimate_cost(self, input_tokens: int, output_tokens: int) -> float:
        pricing = self.PRICING[self.model]
        return (input_tokens * pricing["input"] +
                output_tokens * pricing["output"]) / 1000

    def record_call(self, input_tokens: int, output_tokens: int):
        cost = self.estimate_cost(input_tokens, output_tokens)
        self.spent += cost
        self.call_log.append({
            "cost": cost,
            "timestamp": time.time(),
            "cumulative": self.spent
        })
        return cost

    def select_model(self, task_complexity: str) -> str:
        """Dynamically select cheapest model that can handle task."""
        budget_remaining = self.budget - self.spent

        if budget_remaining < 0.05:  # < 5 cents
            return "claude-haiku-4-5-20251001"
        elif budget_remaining < 0.25 or task_complexity == "simple":
            return "gpt-4o-mini"
        elif task_complexity == "complex":
            return "claude-opus-4-6"
        else:
            return self.model

    def budget_summary(self) -> dict:
        return {
```

```
            "budget": self.budget,
            "spent": self.spent,
            "remaining": self.budget - self.spent,
            "percent_used": (self.spent / self.budget) * 100,
            "total_calls": len(self.call_log),
            "avg_cost_per_call": self.spent / max(len(self.call_log), 1)
        }
```

13.5 Retry & Resilience

python

```
import tenacity
from tenacity import retry, stop_after_attempt, wait_exponential

class ResilientAgent:
    @retry(
        stop=stop_after_attempt(3),
        wait=wait_exponential(multiplier=1, min=4, max=60),
        retry=tenacity.retry_if_exception_type(
            (RateLimitError, APITimeoutError)
        )
    )
    async def call_llm(self, messages: list) -> str:
        """LLM call with automatic retry on transient failures."""
        return await self.llm.ainvoke(messages)

    async def execute_tool_safely(self, tool_name: str, args: dict) -> str:
        """Execute tool with timeout and error handling."""
        try:
            async with asyncio.timeout(30):  # 30 second timeout
                result = await self.tools[tool_name].arun(**args)
                return result
        except asyncio.TimeoutError:
            return f"Tool '{tool_name}' timed out after 30 seconds"
        except Exception as e:
            # Log error and return informative message
            log.error("tool_error", tool=tool_name, error=str(e))
            return f"Tool '{tool_name}' failed: {str(e)}. Try a different approach."

    def run_with_fallback(self, task: str) -> str:
        """Try primary approach, fall back to simpler methods."""
        try:
            return self.primary_agent.run(task)
        except (RateLimitError, BudgetExceededError):
            log.warning("Falling back to secondary agent")
            return self.fallback_agent.run(task)
        except Exception as e:
            log.error("All agents failed", error=str(e))
            return f"I encountered an error: {e}. Please try again."
```

---

14. LATEST RESEARCH (2024–2025)

14.1 Major Breakthroughs

🧠 Test-Time Compute Scaling (2024-2025)

The biggest paradigm shift: spending more computation at inference time enables dramatically better reasoning.

Training compute scaling law: Performance ∝ N^α (N = parameters)
Test-time compute scaling law: Performance ∝ T^β (T = thinking tokens)

OpenAI o3 on ARC-AGI: 87.5% (vs humans: ~84%)
DeepSeek-R1 on AIME 2024: 79.8% (vs o1: 79.2%)

python

```python
# Prompt for extended thinking (Claude)
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-opus-4-5",
    max_tokens=16000,
    thinking={
        "type": "enabled",
        "budget_tokens": 10000  # Allow extensive internal reasoning
    },
    messages=[{"role": "user", "content": "Solve this complex problem..."}]
)

# Access thinking process
for block in response.content:
    if block.type == "thinking":
        print("Claude's reasoning:", block.thinking)
    elif block.type == "text":
        print("Final answer:", block.text)
```

🔬 DeepSeek-R1: RL-Only Reasoning (2025)

First demonstration that reasoning can emerge purely through RL without supervised imitation.

python

```
# GRPO (Group Relative Policy Optimization)
# Key innovation: group sampling instead of value function

def grpo_reward_signal(policy_model, prompt: str, n_samples: int = 8):
    """
    Instead of training a separate value function (PPO),
    use relative rewards within a group of sampled outputs.
    """
    # Sample multiple completions
    outputs = policy_model.sample(prompt, n=n_samples)

    # Score each output
    rewards = [reward_function(prompt, o) for o in outputs]

    # Compute advantages as deviation from group mean
    mean_r = sum(rewards) / n_samples
    std_r = (sum((r - mean_r)**2 for r in rewards) / n_samples) ** 0.5
    advantages = [(r - mean_r) / (std_r + 1e-6) for r in rewards]

    # Key insight: no separate value model needed!
    # The group average IS the baseline
    return list(zip(outputs, advantages))

# Key findings from DeepSeek-R1:
# 1. Models develop "aha moments" - sudden capability jumps
# 2. Reasoning patterns emerge without being taught
# 3. Chain-of-thought length correlates with difficulty
# 4. Self-correction emerges naturally from RL
```

🌐 Agentic RAG (2024)

RAG evolves from passive retrieval to active multi-step retrieval strategies.

python

```
class AgenticRAG:
    """Agent takes control of entire retrieval pipeline."""

    RETRIEVAL_STRATEGIES = {
        "simple": "Single vector search",
        "iterative": "Multiple searches, progressively refined",
        "tree": "Hierarchical document exploration",
        "hypothetical": "Generate hypothetical answer, search for similar",
        "multi_query": "Multiple query formulations, merge results",
        "step_back": "Abstract to higher level, then drill down",
    }

    def retrieve(self, query: str) -> list:
        # 1. Query analysis
        strategy = self.select_strategy(query)

        if strategy == "hypothetical":
            # HyDE: Hypothetical Document Embeddings
            hypothetical_doc = self.llm.invoke(
                f"Write a hypothetical answer to: {query}"
            )
            return self.vector_search(hypothetical_doc)

        elif strategy == "multi_query":
            # Generate 3-5 query variations
            queries = self.generate_query_variations(query, n=4)
            results = []
            for q in queries:
                results.extend(self.vector_search(q))
            return self.deduplicate_and_rank(results, query)

        elif strategy == "step_back":
            # Step-back prompting: go from specific to general
            abstract_query = self.llm.invoke(
                f"What is the broader concept behind: {query}"
            )
            broad_results = self.vector_search(abstract_query)
            specific_results = self.vector_search(query)
            return self.merge_results(broad_results, specific_results)

        elif strategy == "iterative":
            return self.iterative_retrieval(query)

        else:
            return self.vector_search(query)
```

```
    def iterative_retrieval(self, query: str, max_rounds: int = 3) -> list:
        """Iteratively search until sufficient context found."""
        all_results = []
        current_query = query

        for round_num in range(max_rounds):
            results = self.vector_search(current_query)
            all_results.extend(results)

            # Check if we have enough context
            sufficiency = self.llm.invoke(f"""
Is this context sufficient to answer: {query}
Context: {[r.page_content for r in results]}
Answer yes/no and explain what's missing:
""")

            if "yes" in sufficiency.lower():
                break

            # Extract what's missing and reformulate query
            current_query = self.llm.invoke(
                f"Based on missing: {sufficiency}\nNew search query:"
            )

        return self.deduplicate_and_rank(all_results, query)
```

🤖 SWE-agent & OpenDevin (2024)

Software engineering agents that can autonomously solve real GitHub issues.

```
python
```

```
# SWE-agent uses Agent-Computer Interface (ACI)
# Key innovations:
# 1. Custom bash tools designed for LLMs (not humans)
# 2. File viewer with line numbers
# 3. Search tools for code navigation
# 4. Constrained edit operations to prevent errors

SWE_AGENT_TOOLS = {
    "view_file": "View file with line numbers and optional range",
    "search_dir": "Search directory for pattern",
    "search_file": "Search within specific file",
    "edit": "Edit file at specific line range",
    "create_file": "Create new file",
    "execute_bash": "Run bash command",
    "submit": "Submit the solution as patch",
}

# Key finding: Tool design matters as much as model choice
# LLM-friendly tools (clear output, structured, forgiving)
# dramatically improve agent performance
```

🏥 Self-Evolving Agents (Agent Hospital, 2024)

Agents that improve without human annotation.

python

```
class SelfEvolvingAgent:
    """Agent that learns from experience without human labels."""

    def __init__(self, base_agent):
        self.agent = base_agent
        self.experience_buffer = []
        self.improvement_threshold = 0.6  # Quality threshold

    def run_and_learn(self, task: str) -> str:
        result = self.agent.run(task)

        # Self-evaluate output quality
        self_eval = self.self_evaluate(task, result)

        # If quality is low, generate improvement
        if self_eval["score"] < self.improvement_threshold:
            improved_result = self.self_improve(task, result, self_eval)

            # Learn from the improvement
            self.experience_buffer.append({
                "task_type": self.classify_task(task),
                "original": result,
                "improved": improved_result,
                "lessons": self.extract_lessons(result, improved_result)
            })

            # Periodically distill experience into agent
            if len(self.experience_buffer) >= 50:
                self.distill_experience()

            return improved_result

        return result

    def self_evaluate(self, task: str, result: str) -> dict:
        """Agent evaluates its own output."""
        eval_prompt = f"""
Evaluate this response to the task.
Task: {task}
Response: {result}

Score on:
1. Correctness (0-1):
2. Completeness (0-1):
3. Clarity (0-1):
```

```
Return JSON.
"""
        scores = json.loads(self.agent.llm.invoke(eval_prompt))
        scores["score"] = sum(scores.values()) / 3
        return scores

    def distill_experience(self):
        """Distill experience buffer into improved few-shot examples."""
        lessons = [exp["lessons"] for exp in self.experience_buffer]

        # Add best examples as few-shot to agent prompt
        best_examples = self.select_best_examples(self.experience_buffer)
        self.agent.update_few_shots(best_examples)

        self.experience_buffer = []  # Clear buffer
```

14.2 Emerging Trends

Trend 1: Multimodal Agents

python

```
# Agents that see, hear, and act
class MultimodalAgent:
    def __init__(self, vision_llm, audio_model, action_model):
        self.vision = vision_llm    # GPT-4V, Claude Vision, Gemini
        self.audio = audio_model    # Whisper + TTS
        self.action = action_model  # GUI automation

    def perceive(self, screenshot, audio_chunk=None) -> str:
        """Process multimodal inputs."""
        vision_output = self.vision.analyze(screenshot)

        context = f"Screen: {vision_output}"
        if audio_chunk:
            context += f"\nUser said: {self.audio.transcribe(audio_chunk)}"

        return context

    def act(self, instruction: str, screenshot):
        """Generate actions from instruction + visual context."""
        # Identify UI elements and their locations
        elements = self.vision.detect_elements(screenshot)

        action = self.action.plan(instruction, elements)
        return self.execute_ui_action(action)
```

Trend 2: Long-Context & Memory Agents

python

```
# With 1M+ token contexts, new agent patterns emerge
class LongContextAgent:
    def __init__(self, llm_1m_context):
        self.llm = llm_1m_context  # Gemini 1.5 Pro, Claude 3 with 200K

    def analyze_entire_codebase(self, code_files: dict) -> str:
        """With 1M context, analyze entire repos at once."""
        all_code = "\n".join([
            f"# File: {fname}\n{code}"
            for fname, code in code_files.items()
        ])

        # No chunking needed!
        return self.llm.invoke(f"""
I'm providing the entire codebase. Analyze it holistically.
Find: architectural patterns, potential bugs, optimization opportunities.

{all_code}
""")
```

Trend 3: Constitutional AI for Agents

python

```
# Agents with internalized values, not just rules
AGENT_CONSTITUTION = """
CORE VALUES:
1. Helpfulness: Genuinely help accomplish the user's goal
2. Harmlessness: Never cause harm to users, third parties, or society
3. Honesty: Never deceive or manipulate

BEHAVIORAL GUIDELINES:
- When uncertain about potential harm, err on the side of caution
- Prefer reversible actions over irreversible ones
- Request minimal permissions needed for the task
- If goal conflicts with values, refuse and explain why

SELF-CRITIQUE PROCESS:
After planning any action, ask:
- Is this action helpful to the user?
- Could this harm anyone?
- Am I being honest about what I'm doing?
- Are there better alternatives?
"""

class ConstitutionalAgent:
    def __init__(self, llm, constitution: str):
        self.llm = llm
        self.constitution = constitution

    def plan_action(self, context: str) -> dict:
        """Plan with constitutional critique."""
        # Initial plan
        initial_plan = self.llm.invoke(f"Plan next action:\n{context}")

        # Constitutional critique
        critique = self.llm.invoke(f"""
Constitution:
{self.constitution}

Proposed action:
{initial_plan}

Critique this action against the constitution.
What concerns do you have?
How should the action be modified?
""")

        # Revised plan
        revised = self.llm.invoke(f"""
```

```
Original plan: {initial_plan}
Constitutional critique: {critique}
Create a revised plan that addresses all concerns:
""")

        return {"action": revised, "critique": critique}
```

14.3 Research Frontiers

World Models for Agents

```python
```

```
# Agents that can predict consequences before acting
class WorldModelAgent:
    def __init__(self, llm, world_model):
        self.llm = llm
        self.world_model = world_model  # Trained on environment dynamics

    def plan_with_lookahead(self, goal: str, state: dict, depth: int = 3) -> list:
        """Use world model to simulate action consequences."""
        best_plan = []
        best_score = -float('inf')

        # Generate candidate actions
        candidate_actions = self.generate_actions(state)

        for action in candidate_actions:
            # Simulate consequences without actually executing
            simulated_trajectory = self.simulate(state, action, depth)
            score = self.evaluate_trajectory(simulated_trajectory, goal)

            if score > best_score:
                best_score = score
                best_plan = simulated_trajectory.actions

        return best_plan

    def simulate(self, state: dict, action: str, depth: int) -> dict:
        """Predict what would happen if we take this action."""
        predicted_state = self.world_model.predict(state, action)

        if depth <= 0:
            return {"states": [predicted_state], "actions": [action]}

        # Recursively simulate further
        next_action = self.llm.invoke(
            f"Given state {predicted_state}, what's the best next action?"
        )
        deeper = self.simulate(predicted_state, next_action, depth - 1)

        return {
            "states": [state, predicted_state] + deeper["states"],
            "actions": [action] + deeper["actions"]
        }
```

14.4 Key Papers Table (2024-2025)

Paper / System	Key Contribution	Year	Impact
OpenAI o1	Test-time compute; internal CoT	2024	⭐⭐⭐⭐⭐
DeepSeek-R1	RL-only reasoning; GRPO algorithm	2025	⭐⭐⭐⭐⭐
LATS	MCTS for LLM agents	2023	⭐⭐⭐⭐
SWE-agent	LLM-friendly ACI for software engineering	2024	⭐⭐⭐⭐
OpenDevin	Open-source SE agent platform	2024	⭐⭐⭐⭐
Mixture-of-Agents	Ensemble of LLMs > single large LLM	2024	⭐⭐⭐⭐
Agent Hospital	Self-evolving agents without human labels	2024	⭐⭐⭐⭐
SIMA	Generalist embodied agent across 3D worlds	2024	⭐⭐⭐⭐
AgentBench v2	Comprehensive multi-domain evaluation	2024	⭐⭐⭐
OmniACT	Vision-language desktop automation	2024	⭐⭐⭐
τ-bench	Tool-use in realistic scenarios	2024	⭐⭐⭐
Claude 3.7 Thinking	Hybrid thinking models for agents	2025	⭐⭐⭐⭐⭐
WebArena+	Harder web navigation benchmark	2024	⭐⭐⭐
OSWorld	Real OS computer task benchmark	2024	⭐⭐⭐⭐
Agent-FLAN	Fine-tuned open-source agents	2024	⭐⭐⭐
LongAgent	Extend context via agent collaboration	2024	⭐⭐⭐

---

15. KEY PAPERS & RESOURCES

Foundational Papers

- **Chain-of-Thought (2022)**: https://arxiv.org/abs/2201.11903
- **Toolformer (2023)**: https://arxiv.org/abs/2302.04761
- **ReAct (2023)**: https://arxiv.org/abs/2210.03629
- **Reflexion (2023)**: https://arxiv.org/abs/2303.11366
- **Tree of Thoughts (2023)**: https://arxiv.org/abs/2305.10601
- **LLM Agent Survey**: https://arxiv.org/abs/2308.11432

- **LATS (2023)**: https://arxiv.org/abs/2310.04406

Multi-Agent

- **AutoGen (2023)**: https://arxiv.org/abs/2308.08155
- **AgentVerse (2023)**: https://arxiv.org/abs/2308.10848
- **Mixture-of-Agents (2024)**: https://arxiv.org/abs/2406.04692
- **DyLAN (2024)**: Dynamic LLM Agent Network

Software Engineering Agents

- **SWE-agent (2024)**: https://arxiv.org/abs/2405.15793
- **OpenDevin (2024)**: https://arxiv.org/abs/2407.16741
- **SWE-bench (2023)**: https://arxiv.org/abs/2310.06770

Benchmarks

- **AgentBench**: https://arxiv.org/abs/2308.03688
- **WebArena**: https://arxiv.org/abs/2307.13854
- **GAIA**: https://arxiv.org/abs/2311.12983
- **OSWorld**: https://arxiv.org/abs/2404.07972
- **τ-bench**: (2024)

Safety & Alignment

- **Constitutional AI**: https://arxiv.org/abs/2212.08073
- **Agent Safety Survey (2024)**: https://arxiv.org/abs/2406.14455

Reasoning Models

- **DeepSeek-R1 (2025)**: https://arxiv.org/abs/2501.12948
- **OpenAI o1 System Card (2024)**: https://openai.com/research/learning-to-reason-with-llms

Courses & Learning

- **DeepLearning.AI: AI Agentic Design Patterns** (Andrew Ng)
- **Hugging Face Agents Course**: https://huggingface.co/learn/agents-course
- **LangChain Academy**: https://academy.langchain.com
- **CrewAI Docs**: https://docs.crewai.com

Tools & Platforms

Tool	Purpose	URL
LangSmith	Agent observability & tracing	smith.langchain.com
Weights & Biases	Experiment tracking	wandb.ai
E2B	Secure code sandbox	e2b.dev
AgentOps	Agent monitoring	agentops.ai
Helicone	LLM API observability	helicone.ai
Braintrust	LLM evaluation	braintrust.dev
Literal AI	LLM observability	literalai.com

QUICK REFERENCE

AGENT DECISION GUIDE

TASK TYPE	RECOMMENDED APPROACH
Simple Q&A	Single agent + RAG
Multi-step research	ReAct or Plan-Execute
Coding tasks	SWE-agent style + code execution
Complex reasoning	o1/R1 style extended thinking
Creative with review	Critic-Generator loop
Team collaboration	CrewAI or AutoGen multi-agent
Complex workflows	LangGraph stateful graph
Uncertain path	LATS or Tree of Thoughts

MEMORY NEED	SOLUTION
Current session	ConversationBufferMemory
Large knowledge base	Vector store (Chroma/Pinecone)
Learn from past runs	EpisodicMemory + embeddings
Structured data	Database + Text-to-SQL

SCALE REQUIREMENT	APPROACH
Prototype	LangChain + OpenAI
Production API	FastAPI + Redis + async

Enterprise	Semantic Kernel + Azure
Cost-sensitive	Budget-aware model routing
Safety-critical	Multi-layer guardrails + HITL

Cheat Sheet Version: 2025 | Covers research up to March 2025 Models: GPT-4o, Claude Sonnet 4.6, DeepSeek-R1, Gemini 1.5 Pro, Llama 3.1 Frameworks: LangChain 0.3+, LangGraph, AutoGen 0.2+, CrewAI 0.8+, Pydantic AI